ASSIGNMENT 15
Convert DEX to JAR Using Dex2Jar and Analyze Java Bytecode

1. Introduction

Android applications are packaged as APK files, which commonly contain one or more Dalvik Executable (DEX) files. DEX files store application instructions for the Android Runtime. Dex2Jar is a tool that converts DEX bytecode into Java class files packaged in a JAR archive. The generated classes can then be examined with a Java bytecode disassembler or viewer to understand application structure and selected program behavior.

2. Objectives

- Understand the purpose of the classes.dex file in an Android application.
- Extract a DEX file from an authorized APK and convert it to a JAR file using Dex2Jar.
- Inspect generated class files and identify classes, fields, methods, and bytecode instructions.
- Interpret selected instructions and document the results of the static analysis.

3. Requirements

- An APK that you own or have explicit permission to analyze.
- Dex2Jar downloaded from its official project or release source.
- Java Development Kit (JDK) installed on the computer.
- An archive utility such as 7-Zip, or an APK analysis tool.
- A terminal or command prompt and a bytecode viewer such as javap.

Safety note: Perform static analysis only on applications you own or are authorized to assess. Do not run an unknown APK as part of this exercise.

4. Theory

An APK uses a ZIP-based package format and may contain classes.dex, classes2.dex, or additional DEX files when multidex is used. Dex2Jar translates DEX instructions into Java class-file format and places the resulting .class files into a JAR archive. Conversion does not recreate the exact original source code: comments and original variable names may be missing, and obfuscation or unsupported instructions can affect readability.

Java bytecode is a sequence of instructions executed by the Java Virtual Machine. A disassembler such as javap can show instructions, method signatures, fields, and access modifiers. Analysts use this output to understand program logic and to guide further authorized security review.

5. Procedure

Step 1: Prepare the APK

Choose a test APK that you own or have permission to inspect. Make a working copy and preserve the original. Record the APK filename before beginning the analysis.

Step 2: Extract the DEX file

Open the APK with an archive utility and extract classes.dex to a working folder. If the APK contains classes2.dex or other DEX files, record them as well; each can be processed separately.

Step 3: Set up Dex2Jar

Extract the Dex2Jar distribution. Open a command prompt or terminal in the extracted folder and locate the script supplied with your version. Ensure that Java is installed and available from the command line.

Step 4: Convert DEX to JAR

On Windows, a common Dex2Jar command pattern is:

```
d2j-dex2jar.bat classes.dex
```

On Linux or macOS, the script is commonly named d2j-dex2jar.sh. Run the script with the extracted DEX file as input. If required, specify the input and output paths according to the help instructions for the installed version. A successful conversion should produce a JAR file; record any warnings or errors.

Step 5: Verify the generated JAR

Open the output JAR using an archive utility or list its contents with a ZIP/JAR tool. Confirm that it contains .class files and package directories. If conversion fails, check that the DEX file path is correct and that the tool's script and Java installation are available.

Step 6: Inspect the Java bytecode

Use javap from the JDK to display the bytecode of a class in the JAR. Replace the example names below with the actual JAR filename and fully qualified class name:

```
javap -classpath output.jar -c -p package.ClassName
```

The -c option displays bytecode instructions, and -p includes non-public members. A graphical class-file viewer may also be used. Select one class and record its fields, method signatures, and notable instructions.

6. Analysis of Generated Bytecode

Review the output and identify the following elements:

- Class and package: record the package path and class name.
- Fields: note the field names, data types, and access modifiers.
- Methods: note method names, parameter types, return types, and visibility.
- Instructions: identify operations such as loading values, arithmetic, field access, method invocation, conditional branches, and return instructions.
- Control flow: look for conditional jumps and branch targets that represent decisions or loops.
- Security-relevant behavior: note potentially sensitive API calls or hard-coded endpoints for further investigation. A bytecode instruction by itself does not prove that a vulnerability exists.

Example bytecode output (illustrative only):

```
public static int add(int, int);
  Code:
     0: iload_0
     1: iload_1
     2: iadd
     3: ireturn
```

Interpretation: iload_0 and iload_1 load the two integer arguments onto the operand stack. iadd adds them, and ireturn returns the integer result. Actual output depends on the class being examined.

7. Observation Table

Item	**Details to Record**
APK file	Name of authorized APK
DEX file	classes.dex / multidex filename
Conversion	Success, warning, or error
Output JAR	Generated JAR filename
Class inspected	Package and class name
Bytecode finding	Method/instruction and interpretation

8. Result

The DEX file of an authorized Android application was converted into a JAR archive using Dex2Jar, and the generated Java class bytecode was inspected. The class structure and selected instructions were analyzed. Complete the observation table with the actual output obtained during the practical.

9. Precautions

- Analyze only APK files that you own or have permission to examine.
- Keep the original APK unchanged and work on a separate copy.
- Do not execute an unknown APK during static analysis.
- Use the command and script names appropriate to the installed Dex2Jar version.
- Remember that conversion may not recover the original source code exactly.
- Do not disclose credentials, personal data, or proprietary code found during analysis.

10. Conclusion

Dex2Jar enables an analyst to convert an Android DEX file into a JAR archive for inspection with Java bytecode tools. Examining the generated class files helps reveal application structure and selected program logic. The output supports authorized static analysis, but any suspected security issue should be validated with additional evidence.

11. References

- Dex2Jar project documentation and release instructions.
- Oracle JDK documentation for the javap disassembler.
- Android Developers documentation on APK packaging and DEX files.